# 단계 3. 구조화 자료 매핑 how 보완

## 벤치마킹한 문헌의 원리와 우리 구현 과정

2026.10.30 · 문경민

# 1. 이번에 바뀐 점

## 핵심

- 문헌 표기를 바로잡았다. OntoDSMS 는 Zhou 외(2023)가 만든 댐 안전 모니터링 온톨로지의 이름이고, 관계형 DB 를 온톨로지에 연결한 기술은 기존 오픈소스 플랫폼 D2RQ 이다. 우리가 벤치마킹한 것은 D2RQ 의 매핑 원리 4 개이다(부록 A).
- how 를 채웠다. 승인 표의 504 행 하나를 따라가며 식별자 → 클래스 → 속성 → 근거·출처 순서로, 각 과정이 무엇을 어떻게 만들고 결과물이 무엇인지 보였다(2 장, 그림 1~4).
- 원리와 다르게 구현한 부분은 데이터 구조에 맞춘 응용 5 가지로 정리했다(3 장). 매핑을 검토하며 찾은 온톨로지의 부족한 점은 검토할 점으로 남겼다(4 장).

## 표 1. 10/16 자료와 비교

| 항목 | 10/16 자료 | 이번 |
|---|---|---|
| **문헌 표기** | "OntoDSMS(Zhou 2023)는 … 관계형 DB 의 자료를 온톨로지에 연결한다" — OntoDSMS 를 기술 이름처럼 씀 | 논문(Zhou 외 2023) · 온톨로지(OntoDSMS) · 연결 도구(D2RQ)를 구분. 벤치마킹 대상 = D2RQ 의 매핑 원리 |
| **문헌의 방법** | 표 → Class, 열 → 속성, 행 → 개체 (대응 원칙만) | 논문 그림 3 의 매핑 구문을 근거로 원리 4 개: 키 값으로 식별자, 표 → 클래스, 열 → 속성, 대응은 사람이 결정 (부록 A) |
| **우리 구현** | Class 별 결과 예시 표 | 한 행을 따라가는 과정 5 개와 과정별 결과물 (2 장) |
| **원리와 다른 부분** | "D2RQ 도구는 쓰지 않음" 한 줄 | 데이터 구조에 맞춘 응용 5 가지와 그 이유 (3 장) |
| **검토할 점** | 없음 | approvalContent 에 세 열이 합쳐짐, ID 접두어 표기 (4 장) |
| **10/19 질문** | — | 매핑이 어떻게 작동하나 → 2 장 · AP 와 APR → 4 장 2 · 댐 기본 자료 → 2.3 |

## 2. 우리 구현: 승인 표 한 행이 KG 개체가 되는 과정

단계 3 은 정형 표(승인 기록, K-water 측정자료)의 값을 해석을 더하지 않고 Ontology 의 개체와 속성으로 옮기는 단계이다. 아래는 승인 표의 504 행 하나를 따라간다. 각 과정 제목의 "원리 n"은 부록 A 의 D2RQ 원리 번호이고, 그림의 노란 점선 상자는 그 원리에 해당하는 D2RQ 구문이다.

### 2.1 입력: 승인 표와 열의 역할

입력은 전처리를 마친 승인 표(approval_records_clean.csv)에서 대상 4 개 댐(충주, 소양강, 횡성, 광동)의 80 행이다. 한 행은 승인 한 건이다. 변환하기 전에 열마다 어떤 역할을 할지 먼저 정했다(원리 4: 대응은 사람이 정한다). 표 2 의 머리 색이 역할이다.

### 표 2. 승인 표(일부)와 열의 역할

| 순차번호 | 관측소코드 | 관측소명 | 승인년월일시분 | 방류시작시간 | 접수방류량 | 비고 |
|---|---|---|---|---|---|---|
| 473 | 1006110 | 횡성 | 2011-06-24 | 2011-06-24 18:00 | 100 | 수문방류 승인 |
| **504** | **1003110** | **충주** | **2011-06-27** | **2011-06-27 18:00** | **2000** | **(실제방류는 20 시부터)** |
| 574 | 1003110 | 충주 | 2011-07-01 | 2011-07-01 18:00 | 2000 | (실제 방류는 19 시부터) |
| 식별자 키 (과정 1) | 관계에 사용 (단계 5) | 라벨 (과정 2) | 속성 하나 (과정 3) | 세 열을 내용 속성 하나로 (과정 3) | | |

표에는 이 밖에 원본 CSV 의 행 번호(row_number_in_file)와 출처 열 6 개(기관, 내려받은 주소, 원본 파일명, SHA-256 등)가 있다. 이 열들은 과정 4·5 의 근거 개체와 출처 기록에 쓴다.

### 2.2 과정 1~5

**과정 1. 식별자 만들기 (원리 1)** 틀 "APR:" + {순차번호}에 504 를 넣어 승인 개체 APR:504 를 만들고, 같은 행에서 틀 "EVI:AP:" + {순차번호}로 근거 개체 EVI:AP:504 를 만든다. 순차번호를 키로 쓴 이유는 원본 자료가 승인 한 건마다 매긴 번호이고, 전처리(단계 2)에서 중복 행을 지운 뒤에는 표 안에서 유일하기 때문이다. 그래서 80 행은 서로 다른 ID 80 쌍이 된다. ID 가 값에서 결정되므로 KG 를 다시 만들어도 같은 ID 가 나오고, 단계 5 는 이 ID 로 개체를 정확히 찾아 관계를 잇는다.



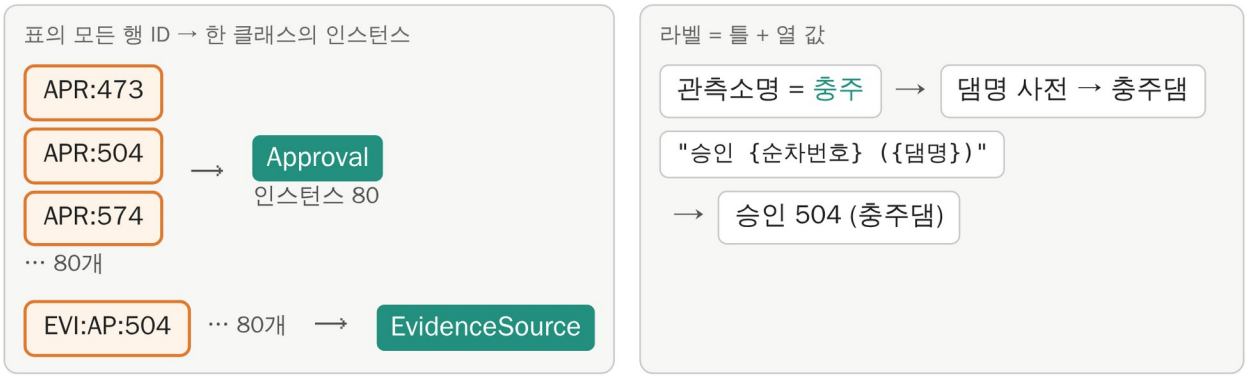

| 순차번호 | 승인 개체 ID | 근거 개체 ID |
|---|---|---|
| 473 | APR:473 | EVI:AP:473 |
| 504 | APR:504 | EVI:AP:504 |
| 574 | APR:574 | EVI:AP:574 |
| ⋯ 80행 → 서로 다른 ID 80쌍 | | |

· 행 하나 → 승인 ID 하나 + 근거 ID 하나
· 같은 행이면 언제 다시 만들어도 같은 ID
· 단계 5는 이 ID로 개체를 찾아 관계를 잇는다

원리 1 · D2RQ 구문 `d2rq:uriPattern "item/@@item.item_ID@@"`
@@ 사이 열(item_ID)의 값을 행마다 끼워 식별자를 만든다: item_ID = 7 → `item/7`

그림 1. 식별자 만들기 — 키 열의 값을 틀에 끼운다

**과정 2. 클래스 지정과 라벨 (원리 2, 원리 3 의 pattern)** 승인 표에서 나온 APR: ID 80 개는 모두 Approval 클래스의 인스턴스로, EVI:AP: ID 80 개는 모두 EvidenceSource 클래스의 인스턴스로 기록한다. 라벨은 틀 "승인 {순차번호} ({댐명})"으로 만든다. 승인 자료의 관측소명은 "충주"이고 K-water 자료는 "충주댐"이라, 관측소명을 표준 댐명 사전으로 바꾼 뒤 틀에 넣는다.




원리 2 · D2RQ 구문 `map:item a d2rq:ClassMap; d2rq:class :Item.`
라벨: `d2rq:property rdfs:label; d2rq:pattern "item #@@item.item_ID@@"`

그림 2. 클래스 지정과 라벨

결과물 (kgv4_entities.csv: 개체 ID, 클래스, 라벨)

```
APR:504,    Approval,       승인 504 (충주댐)
EVI:AP:504, EvidenceSource, 한강홍수통제소 댐방류승인 순차번호 504
```

**과정 3. 열 → 속성 (원리 3)** 승인년월일시분 열은 approvalTime 속성에 대응한다. 원본 값에 시각이 없으므로 앞 10 글자를 날짜(date)로 저장한다(column). 방류시작시간, 접수방류량, 비고 세 열은 문자열 틀에 넣어 approvalContent 하나로 저장한다(pattern). 주어진 온톨로지에 Approval 의 내용 속성이 approvalContent 하나뿐이어서 이렇게 모았다(4 장 검토할 점 1). 관측소코드는 속성으로 만들지 않고, 단계 5 에서 Dam 개체를 찾아 concernsDam 관계로 잇는 데 쓴다.




그림 3. 열 → 속성 — 원본 셀 값, 변환 규칙, 결과

결과물 (kgv4_properties.csv: 개체 ID, 속성, 값, 자료형)

```
APR:504, approvalTime,    2011-06-27, date
APR:504, approvalContent, 방류시작 2011-06-27 18:00; 접수방류량 2000㎥/s; 비고: (실제방류는 20시부터),
string
```

**비고는 어떻게 다루나** 이 단계에서 비고는 표의 한 칸으로서 셀 값을 그대로 옮기기만 한다. 앞뒤 공백을 지우고, 비어 있으면 넣지 않고(80 행 중 15 행), 비어 있지 않으면 "비고: " 뒤에 원문을 붙인다. "(실제방류는 20시부터)"가 실제 방류 시각이라는 것을 읽어 운영행위 OPR:504:1(방류, 20:00)을 만드는 것은 단계 4(비정형 텍스트 추출)의 일이다. 같은 열이 두 단계에 나오지만, 단계 3 은 원문을 보존하고 단계 4 는 그 원문을 해석한다. 원문이 승인 개체에 남아 있어서 단계 4 의 추출 결과를 원문과 바로 대조할 수 있다.

**과정 4. 근거 개체 (응용 1, 3 장)** 같은 행의 출처 열을 근거 개체 EVI:AP:504 의 속성으로 옮긴다. 승인 개체와 근거 개체는 같은 행에서 함께 만들어지므로, 다른 표를 찾지 않고 바로 APR:504 —supportedBy→ EVI:AP:504 로 잇는다.

```
EVI:AP:504, sourceTitle,   환경부 한강홍수통제소 홍수예보 댐방류승인
EVI:AP:504, sourceType,    승인자료
EVI:AP:504, sourceLocator, 순차번호 504
```

**과정 5. 출처 기록 (응용 1, 3 장)** 개체마다 원본 파일, 원본 안의 위치, 적용한 매핑 규칙의 ID, 값의 성격을 한 줄로 남긴다. row_number_in_file 은 원본 CSV 의 행 번호라서 원 파일의 해당 행을 바로 열 수 있고, rule 은 어떤 규칙으로 만들었는지(D2RQ 에서 매핑 파일이 하던 기록), real 은 원본 값을 가공 없이 옮겼다는 뜻이다.

```
개체    원본 파일           위치           규칙   값의 성격  메모
APR:504  approval_records_clean.csv  row_number_in_file=504  MR-APR  real       승인일은 날짜만 있음
```

**결과** 504 행 하나에서 승인 개체와 근거 개체가 속성을 갖추고 supportedBy 로 이어진다. 댐과의 관계(concernsDam)와 운영행위(OPR:504:1)는 이후 단계에서 만든다.

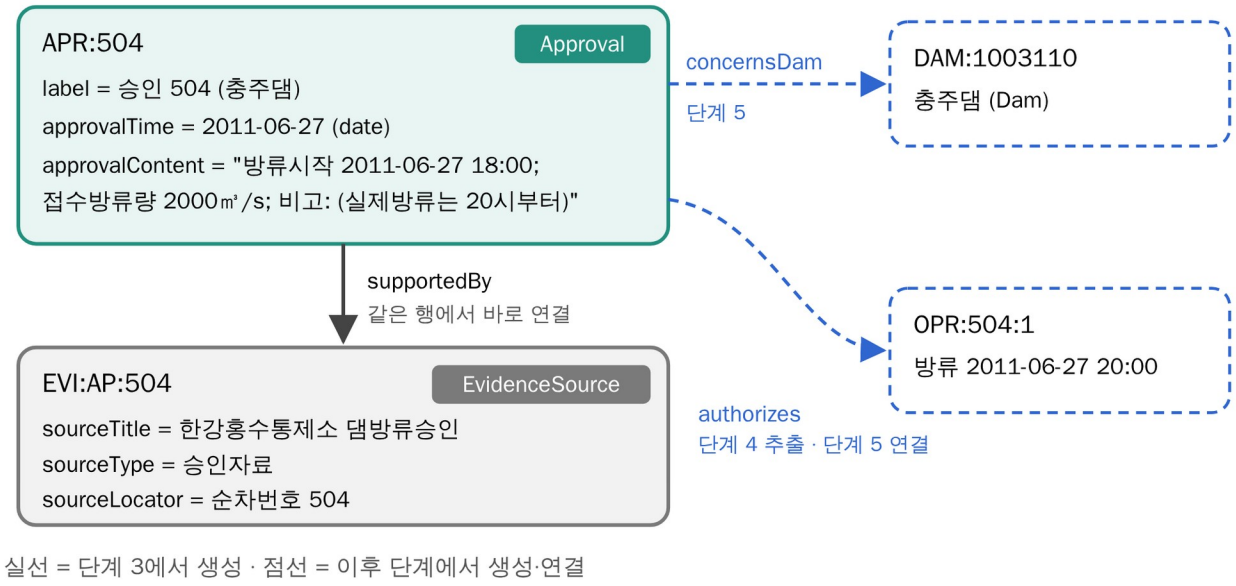



그림 4. 504 행 하나에서 만들어진 그래프 조각

## 2.3 댐 표와 측정 표

**Dam (댐 기본 자료)** 댐 한 개가 한 행인 기본 정보 표는 따로 없고, K-water 일자료(kwater_mywater_period_daily_long.csv.gz)의 측정 행마다 댐 코드(DAM_CD)와 댐 유형이 반복해서 붙어 있다. 틀 "DAM:" + {DAM_CD}로 식별자를 만들면 같은 댐의 여러 행이 같은 ID 로 모이므로, 이를 하나의 Dam 인스턴스로 기록한다. 댐 유형 열은 damType 속성으로 옮기고, damName 에는 표준 댐명을 넣는다. 이 Dam 개체는 단계 5 에서 승인(concernsDam)과 측정 상태(hasHydrometeorologicalState)가 모이는 중심이 된다.

```
DAM:1003110, Dam, 충주댐
DAM:1003110, damName, 충주댐,   string
DAM:1003110, damType, 다목적댐, string
```

**HydrometeorologicalState (HMS)** K-water 시간자료는 한 행이 (댐 코드, 시각)이고 변수 6 개(수위, 저수량, 강우량, 유입량, 총방류량, 저수율)가 열로 펼쳐진 넓은 형태이다(553,609 행). 행의 키(DAM_CD)와 열 머리(변수명)를 함께 틀에 넣어 HMS:1003110:수위처럼 (댐, 변수)마다 개체를 만든다(댐 4 × 변수 6 = 24 개). 시각별 값은 KG 에 넣지 않고 측정 저장소에 두며, 질문이 오면 (댐 코드, 시각)으로 조회한다. 같은 방식으로 댐마다 측정자료 근거 개체 EVI:KW:{DAM_CD}(4 개)를 만든다.

K-water 시간자료 (넓은 형태, 553,609행)

| DAM_CD | SDATE | DATA1_수위 | DATA2_저수량 | … | DATA7_저수율 |
|---|---|---|---|---|---|
| 1003110 | 2021010101 | 131.55 | 1484.5892 | … | 53.99 |
| 1003110 | 2021010102 | 131.54 | 1484.0261 | … | 53.96 |
| … 댐 4개 × 시각 | | | | | |

DAM_CD = 1003110 + 열 머리 = 수위 → `"HMS:"+{DAM_CD}+":"+{변수}` → HMS:1003110:수위

· 댐 4개 × 변수 6개 = HMS 개체 24개 · 속성: variableType = 수위, stationCode = 1003110
· 시각별 값(131.55 등)은 KG에 넣지 않고 측정 저장소에 둔다 → 질문이 오면 (댐 코드, 시각)으로 조회

그림 5. K-water 시간자료 → HMS 개체

## 2.4 단계 3 의 결과물

- 개체 192 개: Approval 80, EvidenceSource 84(승인 80 + 측정자료 4), HMS 24, Dam 4
- 속성 468 개: 승인 160, 근거 252, HMS 48, 댐 8
- 개체마다 출처 기록 1 줄

이 단계가 끝나면 개체와 속성은 갖춰지지만, 서로 다른 표에서 나온 개체(승인과 댐 등)는 아직 연결되지 않았다. 연결은 단계 5 에서 다룬다.

## 3. 데이터 구조에 맞춘 응용

원리는 그대로 두고, 우리 자료의 구조가 OntoDSMS 의 DB 와 달라서 구현 방식을 바꾼 부분이다.

**응용 1. 원본이 DB 가 아니라 파일 → 값을 옮기고 출처를 기록** OntoDSMS 의 원본은 MySQL DB 였고, D2RQ 는 원본 DB 를 그대로 둔 채 질의할 때마다 매핑을 거쳐 읽는다(가상 그래프). 우리 원본은 공공데이터 포털과 K-water 에서 내려받은 CSV·JSON 파일이라 질의를 받을 수 있는 DB 가 아니다. 그래서 매핑한 값을 그래프 DB(Neo4j)로 옮겼다. 값을 옮기면 KG 의 값과 원본 행의 연결이 끊기므로 과정 4(근거 개체)와 과정 5(출처 기록)를 추가했다. 같은 이유로 매핑 규칙은 별도 매핑 파일 대신 변환 코드에 적고, 각 개체에 규칙 ID 를 남겼다. 열이 몇 개뿐이라 자동 생성 후 수정하는 단계 없이 처음부터 사람이 대응을 정했다.

**응용 2. 승인 시각 열에 날짜만 있음 → 날짜형으로** 열 이름은 승인년월일시분이지만 원본 값은 모두 날짜뿐이다. 없는 시각을 만들어 넣지 않고 값의 실제 형태대로 date 로 저장했고, 이 사실을 출처 기록의 메모로 남겼다.

**응용 3. 댐 기본 정보 표가 없음 → 같은 키의 여러 행을 하나의 인스턴스로** 원리 2 는 행 하나가 인스턴스 하나이지만, 우리 자료에서 댐 정보는 측정 행마다 반복될 뿐이다. 식별자가 키 값에서 결정되므로(원리 1) 같은 DAM_CD 의 행들은 같은 ID 로 모이고, 이를 하나의 Dam 인스턴스로 기록했다.

**응용 4. 넓은 형태의 측정 표, 55 만 행 → (키 열, 열 머리)로 개체** 원리를 그대로 적용하면 시간자료의 각 행이 인스턴스가 되어 553,609 개의 노드가 속성 6 개씩을 갖는다. KG 에 필요한 것은 "어느 댐에 어떤 측정 변수가 있는가"라는 구조이고, 시각별 값은 측정 저장소에서 시각으로 찾으면 된다. 그래서 키가 들어 있는 두 곳(키 열 DAM_CD 와 열 머리 변수명)을 식별자 틀에 넣었다. 원리 1 을 넓은 표에 맞게 적용한 것이다.

**응용 5. 다른 개체를 가리키는 열 → 속성이 아니라 관계로** 승인 표의 관측소코드 값은 K-water 의 댐 코드와 같다. 이 값을 문자열 속성으로 두지 않고 단계 5 에서 Dam 개체를 찾아 concernsDam 관계로 잇는 데 썼다. 그래서 "충주댐의 승인"을 문자열 비교가 아니라 관계를 따라가서 찾는다. D2RQ 규격에도 이런 열을 다른 클래스로 잇는 선언(refersToClassMap, join)이 있는 것으로 알려져 있으나, 원문은 아직 확인하지 못했다.

## 4. 검토할 점 (온톨로지 발전 후보)

**1. approvalContent 하나에 세 열이 들어가 있음** 방류시작시간, 접수방류량, 비고를 문자열 하나로 저장하고, 접수일자 열은 어느 속성에도 들어가지 않았다. 구축 당시 주어진 온톨로지에서 Approval 의 속성은 approvalTime 과 approvalContent 두 개뿐이었고, 온톨로지는 수정하지 않는다는 전제로 매핑했기 때문이다. 즉 데이터 구조가 아니라 온톨로지의 속성 정의가 합치게 만든 것이다. 세 열은 원본에서 이미 나뉘어 있고 형태(시각, 숫자, 문장)도 분명하므로, 원리 3 대로라면 속성도 따로여야 한다. 합친 결과 "접수방류량 2000㎥/s 이상인 승인", "승인일과 방류 시작의 시간 차" 같은 조건 질의를 할 수 없다.

- 선택지 ① Approval 속성 추가: dischargeStartTime(datetime), requestedDischarge(decimal, ㎥/s), remark(string), 필요하면 receiptDate(date). 추가하면 KG 재생성, 검증, Cypher 내보내기를 다시 한다.
- 선택지 ② 현재대로 두고, 승인 내용은 답변에 원문으로 보여 주는 용도로만 쓴다.
- 판단 기준: 지금의 CQ4(과거 방류 승인)는 댐과 기간으로 승인을 찾아 내용을 그대로 돌려주므로 분리가 꼭 필요하지 않다. 방류량이나 시각을 조건으로 쓰는 질의를 CQ 에 넣을 때 함께 정한다.

**2. ID 접두어 표기 (10/19 질문: AP 와 APR)** APR:504 와 EVI:AP:504 는 같은 행에서 나온 서로 다른 클래스의 개체(승인, 근거)이다. 접두어는 클래스를, 숫자는 같은 원본 행을 나타낸다. 두 개체의 연결은 문자열을 비교해 찾는 것이 아니라, 같은 행을 처리할 때 두 ID 를 함께 만들면서 supportedBy 관계로 바로 잇는다(과정 4). 그래서 연결은 된다. 다만 EVI 안의 AP 는 출처 종류(승인자료)를 뜻하는데 클래스 접두어 APR 과 비슷해 헷갈린다는 지적은 유효하다. 출처 종류 표기를 클래스 접두어와 맞추는 것(EVI:APR:504)과, 현행을 유지하고 ID 규칙을 문서로 정리하는 것 중에서 정한다.

# 부록 A. 문헌 근거: 벤치마킹한 D2RQ 의 매핑 원리

## [변경] 10/16 자료 2.5 절 "문헌의 방법"을 대체

**표기** Zhou, Z. 외 (2023). BIM and ontology-based knowledge management for dam safety monitoring. Automation in Construction, 145, 104649. OntoDSMS 는 이 논문이 구축한 댐 안전 모니터링 시스템(DSMS) 도메인 온톨로지의 이름이다. D2RQ 는 이 논문이 만든 것이 아니라 기존 오픈소스 플랫폼이며, 논문은 MySQL 에 저장된 모니터링 데이터를 OntoDSMS 에 연결할 때 D2RQ 를 사용했다(4 장, 그림 3). 본 연구가 벤치마킹한 것은 D2RQ 의 매핑 원리이다.

논문 4 장에 따르면 D2RQ 는 "표는 클래스, 열은 속성(DatatypeProperty), 행은 인스턴스, 셀은 값"이라는 대응을 매핑 파일(Turtle 문법)의 선언으로 구현한다. 아래는 논문 그림 3(수정된 매핑 파일 일부)의 구문이다.

```
map:item a d2rq:ClassMap;
    d2rq:uriPattern "item/@@item.item_ID@@";
    d2rq:class :Item.

map:item__label a d2rq:PropertyBridge;
    d2rq:belongsToClassMap map:item;
    d2rq:property rdfs:label;
    d2rq:pattern "item #@@item.item_ID@@".

map:item_Genre a d2rq:PropertyBridge;
    d2rq:belongsToClassMap map:item;
    d2rq:property :itemName;
    d2rq:column "item.Genre".
```

**원리 1. 행의 키 값으로 개체 식별자를 만든다** ClassMap 의 uriPattern 은 @@ 사이에 적힌 열(item 표의 item_ID)의 값을 행마다 끼워 넣는 틀이다. item_ID 가 7 인 행은 item/7 이라는 식별자를 갖는다. 식별자가 키 값에서 결정되므로 같은 행은 언제나 같은 식별자를 받고, 다른 매핑이나 질의가 이 식별자로 개체를 다시 가리킬 수 있다.

**원리 2. 한 표에서 나온 식별자는 모두 같은 클래스의 인스턴스가 된다** 같은 ClassMap 의 d2rq:class :Item 은 위 틀로 만든 모든 식별자를 Item 클래스의 인스턴스로 선언한다. 논문은 이를 "Converting a table in a database to a class in an ontology"라고 설명한다.

**원리 3. 열 하나를 속성 하나에 대응시키고, 셀 값을 속성값으로 옮긴다** PropertyBridge 는 값을 어느 개체에 붙일지(belongsToClassMap), 온톨로지의 어떤 속성인지(property), 어느 열의 셀 값을 옮길지(column)를 선언한다. 논문은 이를 "Converting a column in a table to a DatatypeProperty in an ontology"라고 설명한다. 고정 문자열과 열 값을 조합할 때는 pattern 을 쓴다(그림 3 의 rdfs:label).

**원리 4. 열과 속성의 대응은 사람이 온톨로지 용어로 정한다** 매핑 파일은 DB 구조로부터 자동 생성되며, 이때 이름은 DB 의 표·열 이름 그대로이다. 이를 사람이 온톨로지 용어에 맞게 수정한다. 그림 3 에서 Genre 열이 :itemName 속성에 대응된 것이 그 결과이다.

본 연구가 따르지 않은 부분: D2RQ 는 데이터를 복사하지 않는 가상 RDF 그래프로 동작하며, RDF 질의(SPARQL)를 SQL 로 번역해 원본 DB 에서 실행한다. 우리 원본은 DB 가 아니므로 이 방식 대신 값을 옮기고 출처를 기록했다(3 장 응용 1).